Guidelines:

1. Do not create any new objects, files unless and until you have been asked to do so.
2. Create the Necessary tables and populate them with Sample data conforming to the requirements.
3. Follow Coding Standards and Best Practices
    a. Ideal declarations
    b. Appropriate Naming Conventions for Variables, Parameters, Objects
    c. Exception Handling wherever applicable
4. With appropriate Anonymous blocks, Test the Functionalities of the code
5. Include the test cases and document the outcomes in the submission folders.
6. Do not deviate from the instructions, requirements of the document or the guidelines for the practice.
7. Read all the requirements carefully and completely, if there are multiple requirements to be implemented, plan your approach.
8. Structure the Submission file appropriately with meaningful names and relevant necessary extensions and order the files suitably as well.

Basic Business Needs

Blockbuster, officially Blockbuster LLC is a provider of movie rental services. Services were offered primarily at video rental shops, where the members (Customers) of the company can rent/buy a copy of a movie they like.

With the influx of Netflix into the industry, they are facing a huge competition but are also having a loyal fan base which can help them to sustain in the industry.

They have approached you to implement a solution with an Oracle Database and PL/SQL programming constructs to make their business easy and reliable.

The simplified Blockbuster Database provides movie services to its members.

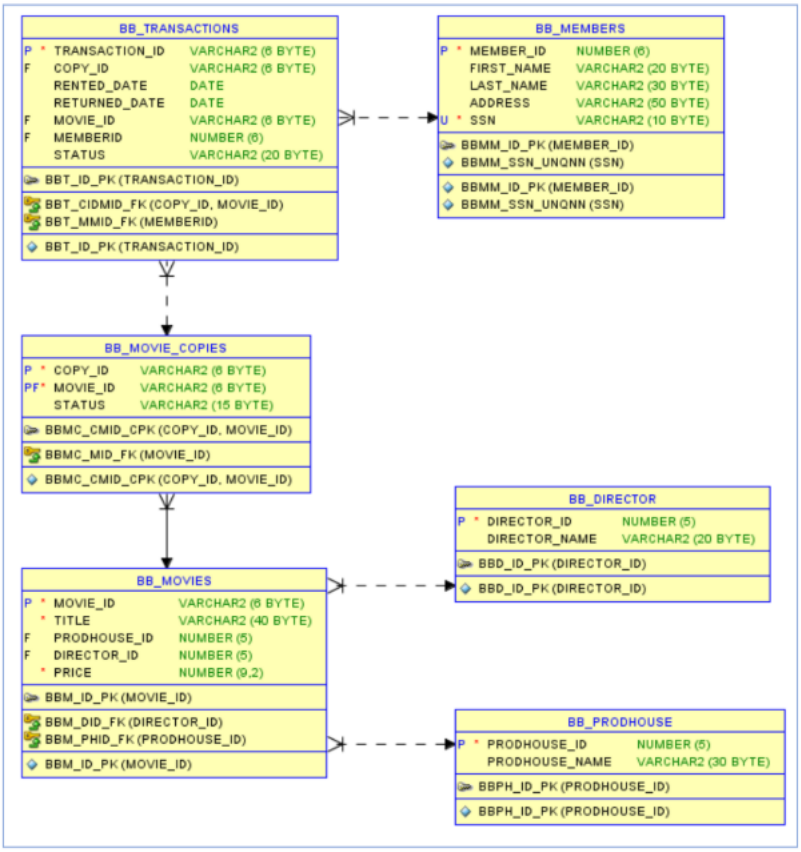
- Blockbuster would like to maintain a library of movies they are currently dealing with.
    - They would like to store the information of Directors, Production Houses and Movies in a structured and appropriate way.
    - They will make copies of a movie and would like to keep a track of the status of that copy.
- Any person can become a member of the Blockbuster store by furnishing some details about themselves.
    - The system will allocate a unique Member_ID for them for their future references.
- Members can collect Movie copies and return them on the specified time.

- This Collection and Returning of the movie copies must be tracked by the system appropriately.

Pre-Requisites

The database contains the following tables:

- BB_Members
- BB_Transactions
- BB_Movie_Copies
- BB_Movies
- BB_Director
- BB_ProdHouse



Functional Requirements:

1. Create a Boolean function (SF_ValidateDirector) which will take Director_ID as input and check whether the value entered is a valid one or not.

Hint: Boolean functions return TRUE or FALSE.

2. Create a Boolean function (SF_ValidateProdHouse) which will take ProdHouse_ID as input and check whether the value entered is a valid one or not.

3. Create a stored procedure (SP_MovieByDirector) which will take Director_ID as input and print the details of all the movies associated with the Director.
   - Do the appropriate validations with the help of function, SF_ValidateDirector.
   - Print all the details of the Movies grouped by the Production House (i.e., once the director is validated, Production House-1 and then movies of that director under that Production House-1, Production House-N and then movies of that director under that Production House-N)

4. Create a stored procedure (SP_NewMember) which will add a new member to the BB_Members table.
   - For the Member ID value, use a sequence SEQ_MemberID and the Member ID should start with 100
   - Pass all other values to be inserted into a new row as parameters

   Do appropriate validations and print the meaningful messages while testing the procedure.
   - Ensure that
     - the First and Last Names of the member are not the same
     - the Address is entered
     - the Social Security Number is valid w.r.t length and uniqueness

5. Create a stored procedure (SP_NewMovie) which will add a new movie to the BB_Movies table
   - For the Movie ID value, use a sequence SEQ_MovieID and the Movie ID should start with 100 and be having "MOV" in the beginning. (Example – MOV100, MOV101, etc.)
   - Pass all other values to be inserted into a new row as parameters
   - Do appropriate validations and print the meaningful messages (Invalid ProdHouse_ID, Invalid Director_ID or Invalid Price [price cannot be less than 500]) while testing the procedure

6. Create a stored procedure (SP_NewMovieCopy) which will add a new movie copy to the BB_Movie_Copies table
   - There can be only 10 non-damaged copies for any movie in the Blockbuster's system.
   - For the Copy ID value, use "MCP" in the beginning. (Example – MCP500, MCP501, etc.)
   - Pass all other values to be inserted into a new row as parameters.
   - Do appropriate validations and print the meaningful messages.

   i. If a copy is being created for a movie which already has 10 copies, it is an invalid scenario, create an exception for the same and handle it.
   ii. The Status should be A (Available) for the movie copy.
   iii. The copy should be made for a valid movie.

7. Create a package named PKG_BBReqs with the following elements:
    - New_Movie_Trns:
        i. An overloaded public procedure to record a new rental.
        ii. Pass the Movie_ID value for the movie that a member wants to rent and either the member's last name or his member ID into the procedure.
        iii. The procedure should check if there are any copies of the movie available for Rental and allocate one copy if available to the member.
        iv. The procedure should return the due date for the movie. Due dates are three days from the date the movie is rented.
        v. Then, insert a new record into the BB_Transactions table identifying the Rented_Date as today's date, the Transaction_ID value using a sequence including a "T" in the beginning and starting with 10000, the member ID, and the movie ID values.
        vi. Be aware of multiple members with the same last name. In this case, display a list of the members' names that match and their ID numbers.
        vii. Once allocated, print appropriate details of the allocation in an Invoice format for the member

    - Return_Movie:
        i. A public procedure that takes Transaction_ID, Copy_ID and Status as inputs
        ii. Ensure that the Copy_ID is validated
        iii. If the Status is given as "Damaged" (D), update it accordingly otherwise it should be set back to Available (A) if no status is passed as input.
        iv. This procedure updates the status of a movie copy (Available/ Damaged) in the BB_Movie_Copies table and sets the returned date in the BB_Transactions table.
        v. Set the Status ("In-Rent", "Damaged", "Over-Timed", "Returned") accordingly in the BB_Transactions table

8. The business hours for the Blockbuster stores are 8:00 A.M. through 10:00 P.M., Sunday through Friday, and 8:00 A.M. through 12:00 P.M. on Saturday.
    - Ensure that the tables can be modified only during these hours.
    - Hint: You can make use of Triggers